

Numbers Blast — Kod ve Sınıf Çalışma Dokümanı

Bu doküman, **Numbers Blast** Unity projesindeki tüm kaynak kodları, sınıfları ve sistemlerin nasıl çalıştığını katman katman anlatan bir eğitim/çalışma kaynağıdır. Amaç; projeyi ilk kez inceleyen bir geliştiricinin, her dosyanın rolünü, sınıfların alanlarını/metodlarını ve sistemlerin birbirine nasıl bağlandığını uçtan uca kavrayabilmesidir.

Proje: Numbers Blast · Block Blast tarzı, sayı birleştirmeli bir bulmaca prototipi

Motor: Unity 6000.3.8f1 (6.3 LTS) · UGUI + yeni Input System

Platform: Android, dikey (Editor'de fare ile de oynanır)

Kod tabanı: 58 C# dosyası, ~5.250 satır, tek assembly (**NumbersBlast**)

1. Oyun Nedir? (Kısa Özet)

Numbers Blast, bir ızgara (board) üzerine sayı taşıyan bloklar (piece) yerleştirilen bir bulmacadır. Temel döngü şudur:

- Yerleştir:** Oyuncu tepsideki (tray) bir parçayı boş hücrelere sürükler.
- Merge (Birleştirme):** Yerleşen bir hücre, kendisiyle **eş değerli** komşularını kendine katar; değer katlanarak büyür ve bu zincirleme olarak devam eder.
- Line Clear (Temizleme):** Tamamen dolan satır/sütunlar temizlenir ve **skor** verir.

Bu çekirdek oynanışın yanında proje, inandırıcı bir **yapay zekâ rakibe** karşı sahte gerçek-zamanlı bir **multiplayer** modu (VS AI) da içerir: 20 saniyelik tur sayacı, ekranda görünür timeout cezası ve tamamen ekran üzerinde oynayan, insansı davranan bir rakip.

2. Mimari Genel Bakış

Proje **tek assembly** (**NumbersBlast**) kullanır ve klasörler namespace'lerle **bire bir** eşleşir. Bağımlılık yönü tek taraflıdır: alt katmanlar (Core, Data, Gameplay) üst katmanları (UI, Input, Presentation) **hiçbir zaman** referans etmez. Bu, saf mantığın test edilebilir ve UI'dan bağımsız kalmasını sağlar.

Katman (klasör)	Namespace	Sorumluluk
Core	NumbersBlast.Core	Değişmez veri yapıları + enum'lar (piece verisi, sonuç yapıları, oyun durumu)
Data	NumbersBlast.Data	ScriptableObject'lar (board yapılandırması, parça şekilleri, tutorial adımları)
Gameplay	NumbersBlast.Gameplay	Saf mantık, UI'sız (board, tray, hamle pipeline'ı, merge, temizleme, skor)
App	NumbersBlast.App	Kompozisyon kökü: GameSessionController + oturum yardımcıları
Presentation	NumbersBlast.Presentation	Görseller, animasyonlar, ortak yerleştirme özizlemesi
Input	NumbersBlast.Input	PieceDragController (fare ve dokunma tek yol)
UI	NumbersBlast.UI	Menü, skorboard, tur/sayaç, tutorial, oyun sonu, eşleştirme
Tutorial	NumbersBlast.Tutorial	3 adımlı zorunlu tutorial
Opponent	NumbersBlast.Opponent	Maç döngüsü, hamle değerlendirici, insansı sunum
Settings	NumbersBlast.Settings	SFX / müzik / titreşim + ayarlar paneli

Assembly yapısı

- NumbersBlast** — Ana assembly. Referansları yalnızca **Unity.TextMeshPro** ve **UnityEngine.UI**. DI framework'ü, service locator veya event bus **yok**.
- NumbersBlast.Tests** — EditMode testleri (yalnızca Editor). NUnit tabanlı.
- NumbersBlast.PlayModeTests** — PlayMode testleri (çalışan oyun üzerinde).

Tek Doğruluk Kaynağı: **PlacementService.ApplyMove**

Projenin en önemli tasarım kararı, **tek bir hamle pipeline'ının** olmasıdır:

```
ApplyMove: yerleştir → merge → temizle
```

Şu dört farklı senaryonun **hepsi** aynı pipeline'ı kullanır:

- Önizleme** — oyuncu parçayı sürüklerken gösterilen tahmin
- Gerçek hamle** — oyuncunun bıraktığı hamle
- AI değerlendirmesi** — rakibin en iyi hamleyi ararken denediği hamleler
- Fail-state kontrolü** — "oynanabilir hamle kaldı mı?" sorusu

Bu sayede gördüğünüz önizleme ile gerçek sonuç **yapısal olarak ayrışamaz**; AI de oyuncuyla **birebir aynı** kurallarla değerlendirilir.

3. Öne Çıkan Tasarım Kararları

- **Önce merge, sonra clear:** Sıralama kuralın bir parçasıdır; önizleme de aynısını gösterir. Skor yalnızca satır/sütun temizliğinden gelir (kesişen hücreler bir kez sayılır).
- **Tray içeriği modeldedir** (**TrayModel**): Parça tüketimi tek noktada ve yalnızca hamle doğrulandıktan sonra yapılır.
- **AI oyuncuyla aynı pipeline'la değerlendirir:** Rubber-band mekaniği maçları yakın tutar, bariz en iyi hamle asla kaçırılmaz; rakip gerçekte oynamayacağı bir hücrede sahte oyalanmaz.
- **Bağımlılık hijyeni:** DI framework'ü, service locator, event bus veya kullanılmayan interface yok — düz, izlenebilir kod.

Aşağıdaki bölümler kodu katman katman, sınıf sınıf ele alır.

4. Core Katmanı (Değişmez Veri + Enum'lar)

Core katmanı oyunun **saf veri/durum modellerini** barındırır: enum'lar, değişmez (immutable) struct'lar ve immutable sonuç sınıfları. Bu tipler bilinçli olarak **değişmez** tutulmuş ve iş mantığından (`PlacementService`, `PieceFactory`) ayrıştırılmıştır — "veri" ile "davranış"ı ayıran temiz bir tasarım. Unity Editor'a bağımlılığı minimumdur (yalnızca `Vector2Int/Color`).

CellHighlightType (enum)

Bir tahta hücresinin sürükleme önizlemesinde ve hamle çözümlenirken gösterebileceği görsel vurgu durumları:

- `None` — vurgu yok
- `ValidPlacement` — geçerli yerleştirme (genelde yeşil)
- `InvalidPlacement` — geçersiz (dolu/tahta dışı; genelde kırmızı)
- `MergeSource` — bu hücre bir birleştirmenin kaynağı olacak
- `LineClear` — bu hücre tamamlanan bir satır/sütun temizliğine dahil olacak

Rol: Tamamen sunum katmanına yönelik sinyal tipi. Sunum, `MoveResult` içindeki `PlacedCells`, `MergeSteps.SourceCells` ve `ClearedCells` verilerini bu enum'a çevirerek her hücreyi renklendirir.

PlayerSide (enum)

Bir hamlenin/tahtanın hangi tarafa ait olduğunu belirtir: `Local` (gerçek oyuncu) ve `Opponent` (VsAI'daki yapay zekâ). `GameMode.VsAI` ve `GameState.PlayerTurn/OpponentTurn` ile doğrudan ilişkilidir.

GameMode (enum)

Oynanış modu: `Solo` (rakipsiz, skor odaklı) ve `VsAI` (yapay zekâya karşı). Üst seviye oyun akışını yönlendirir; `VsAI`'da oyun `PlayerTurn/OpponentTurn` arasında geçiş yapar.

GameState (enum)

Oyunun anlık durumunu ifade eden **sonlu durum makinesi**:

- `MainMenu` — ana menü, oyun başlamadı
- `PlayerTurn` — yerel oyuncunun hamle sırası
- `OpponentTurn` — rakibin (AI) sırası
- `ResolvingMove` — hamle uygulanıyor (yerleştirme/merge/temizleme animasyonları); girdi kilitlenir
- `GameOver` — oyun bitti

Rol: Merkezi kontrol akışının belkemiği. Girdi kabulü, animasyon oynatımı ve tur geçişleri bu duruma göre yönetilir. **ResolvingMove**, oyuncunun çözümleme sırasında yeni hamle yapmasını engeller.

PieceCellData (readonly struct)

Bir parçanın tek hücrelerini temsil eder — değişmez değer tipi:

- **Vector2Int Offset** — çapaya (anchor) göre yerel grid ofseti ((0,0) çapanın kendisi)
- **int Value** — hücrenin sayısal değeri (birleştirme mekaniğinin temeli)

Yorum: "Immutable so pieces cannot mutate after creation." **readonly struct** seçimi hem değişmezliği garanti eder hem de heap tahsisi yapmadan verimli kopyalanır. Bir parça birden çok **PieceCellData**'dan oluşur; şekil (**PieceShapeDefinition**) + değer (spawn'da atanır) = liste.

MergeStep (sealed class, immutable)

Hamle çözümlemesindeki **tek bir birleştirme olayını** temsil eder:

- **Vector2Int Target** — birleştirilmiş değeri alan hedef hücre
- **int OriginalValue** — birleşme öncesi ortak değer
- **int ResultValue** — birleşme sonrası yeni değer
- **IReadOnlyList<Vector2Int> SourceCells** — birleşen tam bağlantılı aynı-değerli grup (**Target dahil**)

Rol: Birleşmenin "kaydını" tutar. Bir hamle birden çok birleşme (zincir) tetikleyebildiği için **MoveResult.MergeSteps** bir liste tutar. **OriginalValue/ResultValue** ayrımı, animasyonda "önce/sonra" gösterimini mümkün kılar.

LineClearResult (sealed class, immutable)

Satır/sütun temizliği sonucunu taşır: **IReadOnlyList<Vector2Int> ClearedCells** (temizlenen hücreler) ve **int ScoreGain** (kazanılan puan). Line-clear hesaplayıcının ara dönüş değeridir; sonuçları nihai **MoveResult**'a aktarılır.

MoveResult (sealed class, immutable) — merkezi sonuç nesnesi

Tek bir hamle çözümleme geçişinin (yerleştirme + merge + temizleme) **eksiksiz sonucunu** taşır.

Yorum: **"Produced by the one central pipeline in PlacementService.ApplyMove, and shared by preview, real moves and opponent evaluation."**

Property'ler:

- **bool IsValid** — hamle geçerli mi
- **IReadOnlyList<Vector2Int> PlacedCells** — yerleştirilen hücreler
- **IReadOnlyList<MergeStep> MergeSteps** — tetiklenen birleştirmeler (sıralı)
- **IReadOnlyList<Vector2Int> ClearedCells** — temizlenen hücreler
- **int ScoreGain** — kazanılan puan

- `bool HasMerge => MergeSteps.Count > 0` ve `bool HasLineClear => ClearedCells.Count > 0` (kısayollar)

Optimizasyon: `static readonly EmptyCells / EmptySteps (Array.Empty<>)` ile boş sonuçlarda tahsis yapılmaz — AI çok sayıda hamle değerlendirirken önemli. Yapıcı null-korumalıdır (`placedCells ?? EmptyCells`), böylece property'ler asla `null` dönmez. ``static MoveResult Invalid()`` fabrikası geçersiz hamle için tek-tip, tahsissiz sonuç üretir.

Rol: Oyun mantığının kalbindeki veri sözleşmesi. Preview, gerçek hamle ve AI değerlendirmesi hep bu tek tip üzerinden çalışır.

5. Data Katmanı (ScriptableObject'ler)

Bu katmandaki tüm tipler `ScriptableObject`'tir: Unity Editor'da **asset** olarak oluşturulup Inspector'dan tasarımcılar tarafından ayarlanır ([`CreateAssetMenu`] ile "Numbers Blast" menüsü altında). Ortak desen: **private serialized field + public getter** — Inspector düzenlenebilirliği ile çalışma-zamanı değişmezliğini birleştirir.

BoardConfig (ScriptableObject) — Numbers Blast/Board Config

Tahtanın boyut, görünüm ve bazı kural parametrelerini tutan merkezi yapılandırma.

Alanlar: `width = 8`, `height = 8`, `trayPieceCount = 3`, `cellSize = 96f`,

`timeoutPenaltyPercent = 0.05f` (tur zaman aşımında kaybedilen skor yüzdesi, %5),

`emptyCellColor` (koyu lacivert), `Color[] valueColors` (değer→renk paleti, indeks 0 = değer 1).

Public getter'lar: `Width`, `Height`, `TrayPieceCount`, `CellSize`, `TimeoutPenaltyPercent`, `EmptyCellColor`.

`GetColorForValue(int value)` — değeri renge çevirir:

1. Palet boş/null → `Color.white`
2. Değer `1..N` aralığında → `valueColors[value - 1]`
3. Değer paleti aşarsa (merge'ler değeri 6, 12, 24... yükseltir) → **altın oran hue döngüsü**:

```
int steps = Mathf.Max(1, value - valueColors.Length);
float hue = (steps * 0.6180339887f) % 1f;
return Color.HSVToRGB(hue, 0.62f, 0.80f);
```

Altın oran (`0.618...`) ardışık değerlerin renk çemberinde birbirine yapışmadan dağılmasını sağlar; deterministiktir ("same value → same colour every time"). `S=0.62`, `V=0.80` bantları üzerine yazılan beyaz metnin okunur kalması için ayarlıdır.

`OnValidate()` — Editor'da değerleri güvenli aralığa sıkıştırır (clamp): `width/height` →

`[4,12]`, `trayPieceCount` → `[1,3]`, `cellSize` ≥ 16, `timeoutPenaltyPercent` → `[0,1]`.

PieceShapeDefinition (ScriptableObject) — Numbers Blast/Piece Shape

Bir parçanın **şekli**ni yerel grid ofsetleri kümesi olarak tanımlar. Kritik not:

"Values are not stored here; they are assigned at spawn time by PieceFactory." Değerler burada yok; spawn'da atanır. Alan: `Vector2Int[] offsets`; getter: `IReadOnlyList<Vector2Int> Offsets`.

Şekil/değer ayrımı: Aynı şekil her spawn'da farklı sayı değerleri alabilir. `PieceFactory` ofsetleri alıp değer atayarak `PieceCellData` üretir.

PieceSetConfig (ScriptableObject) — Numbers Blast/Piece Set

Kullanılabilir tüm şekil havuzunu ve spawn değer aralığını tanımlar.

Alanlar: `PieceShapeDefinition[] shapes` (havuz), `minValue = 1`, `maxValue = 4`.

Getter'lar: `Shapes`, `MinValue`, `MaxValue`.

Tasarım kısıtı: aralık genişletilirken en büyük şeklin hücre sayısı kadar farklı değer olmalı; aksi halde `PieceFactory`'nin "komşu hücrelerde aynı değer olmasın" garantisi bozulur ve fabrika açgözlü (greedy) en-iyi-çaba atamasına düşer. `OnValidate()`: `minValue → [1,9]`, `maxValue → [minValue, 9]`.

TutorialStepDefinition (ScriptableObject) — `Numbers Blast/Tutorial Step`

Tek bir etkileşimli tutorial adımını tanımlar. Not: tahta ve parça preset'leri, ayrı SO'lar yerine serileştirilebilir struct'lar olarak **gömülüdür** (veri ayak izini küçük tutmak için).

Ana sınıf alanları: `instructionText ([TextArea])`, `BoardCellPreset[] boardCells`

(başlangıç tahtası), `TutorialPiecePreset[] trayPieces` (sunulan parçalar),

`Vector2Int[] allowedAnchors` (izin verilen yerleştirme konumları), `Vector2Int[] highlightedCells`

(vurgulanan hücreler). Hepsi salt-okunur getter'larla dışa açılır.

Yardımcı struct'lar ([Serializable]):

- `BoardCellPreset { Vector2Int Position; int Value; }` — önceden kurgulanmış tahta hücresi.
- `TutorialPiecePreset { PieceCellPreset[] Cells; }` — garantili parça tanımı.
- `PieceCellPreset { Vector2Int Offset; int Value; }` — Core'daki `PieceCellData`'nın **editör-zamanı** eşdeğeri (çalışma zamanında `PieceCellData`'ya dönüştürülür).

Rol: Tutorial sistemi bir dizi `TutorialStepDefinition` üzerinden ilerler; her adımda tahtayı `BoardCells`'e göre kurar, tepsiyi `TrayPieces` ile doldurur, `AllowedAnchors` ile doğru hamleyi zorlar, `HighlightedCells` ile dikkat yönlendirir. Öğretimi tamamen **data-driven** yapar.

6. Gameplay Katmanı (Saf Mantık — UI'sız)

Gameplay katmanının tüm sınıfları **Unity'den bağımsız** (MonoBehaviour olmayan) "saf model/servis"lerdir;

bu sayede test edilebilir, kopyalanabilir ve simüle edilebilir. Ana felsefe: **tek doğruluk kaynağı** olan `PlacementService.ApplyMove` — hamle çözümlemesini **yerleştir** → **birleştir** → **satır temizle** sırasıyla yapar. Önizleme, gerçek hamle ve rakip değerlendirmesi hep bu metodu kullanır.

BoardModel

`public sealed class BoardModel` — saf tahta durumu, MonoBehaviour **değil** (*"Knows nothing about

Unity views/UI"*).

Alanlar: `int[] _cells` (2B ızgara → **düz/flat tek boyutlu dizi**; 0 = **boş hücre**), `int Width`, `int Height`.

Metodlar:

- `BoardModel(int width, int height)` — `_cells = new int[width*height]` (C# int dizisi 0 ile başlar → boş tahta).
- `IsInside(pos)` — `x>=0 && x<Width && y>=0 && y<Height` (tüm erişimlerin güvenlik bekçisi).
- `IsOccupied(pos)` — sınır içi VE değeri ≠ 0.
- `GetValue(pos)` — sınır içiyse değeri, dışıysa 0 döner (algoritmalar sınır dışını "boş" gibi ele alır).
- `SetValue(pos, value) / Clear(pos)` — yalnız sınır içiyse yazar/siler (savunmacı).
- `ClearAll()` — tüm hücreleri 0 yapar (yeniden başlama).
- `CopyFrom(source)` — **simülasyon için kritik**. `Array.Copy` ile başka tahtanın durumunu kopyalar ("the real board is never mutated during simulation").
- `ToIndex(pos)` — `y * Width + x` (**satır-öncelikli / row-major** yerleşim).

Rol: Tüm pipeline'ın üzerinde çalıştığı veri zemini. `CopyFrom` sayesinde aynı çözümleme hem gerçek hem "scratch" kopyada çalışır — tek doğruluk kaynağını mümkün kılan temel.

TrayModel

`public sealed class TrayModel` — tepsideki parçaların **yetkili listesi** (view'dan değil, session'dan).

Alanlar: `List<PieceInstance> _pieces`, `ReadOnlyList<PieceInstance> Pieces` (salt-okunur görünüm), `bool IsEmpty`.

Metodlar: `SetPieces(pieces)` (Clear + tek tek ekle), `Remove(piece)` (yerleştirilen parça düşer), `Clear()`.

Rol: `HasAnyValidMove` (fail-state) ve rakip değerlendirmesi için girdi kaynağı. View'dan bağımsız yetkili kaynak olduğu için simülasyonda tutarlı — "sızan tek MVC dikişini" kapatan düzeltme.

PieceInstance

`public sealed class PieceInstance` — değişmez somut parça. Alan: `ReadOnlyList<PieceCellData> Cells`

(her eleman `Offset + Value`). Tek kurucu `PieceInstance(cells)`. Kasıtlı "aptal veri taşıyıcısı".

Rol: Oyuncunun taşıdığı atomik birim. `CanPlace/ApplyMove` her hücrenin `anchor + Offset` konumunu hesaplar. Değişmezlik, aynı parçanın önizleme + gerçek hamlede yan etkisiz kullanımını sağlar.

PieceFactory

`public sealed class PieceFactory` — `PieceSetConfig`'ten rastgele parça üretir. Kritik kural: *bir parça,

konur konmaz kendi içinde birleşemez — komşu iki hücre asla aynı değeri almaz (spawn'da self-merge yok).*

Alanlar: `const int MaxAttempts = 32, static Vector2Int[] Directions` (4-yön), `_config, System.Random _random`

(Unity değil — **tohumlanabilir/deterministik**), `_minValue, _maxValue`.

Metodlar:

- `PieceFactory(config, int? seed = null)` — seed varsa deterministik RNG (replay/test için).
- `CreateRandomPiece()` — rastgele şekil seçer, `CreatePiece`'e verir.
- `CreatePiece(offsets)` — 32 kez `TryAssignRandom`; başarısızsa `AssignGreedy` (garantili fallback).
- `TryAssignRandom(...)` — her offset için yasak olmayan aday değerler toplanır, biri rastgele seçilir; hiç aday kalmazsa `false` (geri izleme yok, bu yüzden 32 deneme).
- `AssignGreedy(offsets)` — her offset'e yasak olmayan **ilk** (en küçük) değeri atar — deterministik güvenlik ağı.
- `IsForbidden(assigned, offset, value)` — 4 komşudan biri zaten `value` ise `true` (self-merge engeli).

Rol: Pipeline'ın başlangıcı — tepsiye konacak parçaları üretir. "Komşu aynı değer olmasın" kuralı, birleşmenin yalnızca bilinçli yerleştirmeye tetiklenmesini garanti eder.

ScoreService

`public sealed class ScoreService` — tek skor değerini yönetir (Part 1: bir örnek; Part 2: yerel + rakip).

Alanlar: `int _score, int Score` (salt-okunur), `event Action<int> ScoreChanged` (UI abone olur — observer deseni).

Metodlar:

- `Add(amount)` — `amount <= 0` yok sayılır; ekler ve `ScoreChanged?.Invoke(_score)`.
- `ApplyPenaltyPercent(percent)` — `penalty = FloorToInt(_score * percent), _score = Max(0, _score - penalty)` (asla 0 altına inmez). Örn. 47'nin %5'i → `floor(2.35)=2`.

- **Reset()** — skoru 0'a çeker, olayı tetikler.

Rol: Hamle sonucunun tüketicisi. Skor mantığı burada değil kaynağında (LineClearResolver) hesaplanır; bu servis yalnızca biriktirir, ceza uygular ve bildirir.

MergeResolver — birleştirme algoritması

public sealed class MergeResolver — aynı değerli hücrelerin birleşmesini deterministik çözer. Kurallar:

4-yön komşuluk; birleşen hücre grubun başlangıcını hedef olarak korur ve **zincirleme** devam eder.

Resolve(board, startPositions) — **ana metod** (startPositions genelde yeni konan hücreler → sadece değişen bölge):

1. Her **start** için hücre boşa atla.
2. **current = start, while (IsOccupied(current))** döngüsü (zincirlemeyi sağlar).
3. **value = GetValue(current); FindConnectedSameValue** ile BFS grubu bulunur.
4. **group.Count < 2** ise **break** (birleşecek komşu yok).
5. Birleşme: **resultValue**, gruptaki tüm değerlerin **toplamı** (üç 3 → 9).
6. Grubun **current dışındaki** hücreleri **Clear**; **current** hedef olarak korunur.
7. **SetValue(current, resultValue); steps.Add(new MergeStep(...))**.
8. **while** tekrar döner: yeni değer başka komşuyla birleşebilir → **zincir** (ta ki komşu kalmayana dek).

FindConnectedSameValue(board, start, value) — **BFS**: **group, visited** (HashSet), **queue**. 4 komşu gezilir; dolu ve değeri eşitse kuyruğa eklenir. HashSet sonsuz döngüyü önler.

Rol: **ApplyMove**'un ikinci aşaması. Toplama-tabanlı, zincirlemeli, deterministik birleştirme — önizleme/AI'nın gerçek hamleyle birebir uyuşması için şart.

LineClearResolver — satır temizleme algoritması

public sealed class LineClearResolver — birleşmeler bitince dolu satır/sütunları temizler. İncelik: kesişen hücreler **HashSet** ile **tek kez** sayılır (puan iki kez eklenmez). Tamamen durumsuz (stateless).

Resolve(board) — **ana metod**:

1. **cellsToClear = HashSet<Vector2Int>()** (tekrarları otomatik eler).
2. **Satır taraması**: her **y** için **IsRowFull** doğruysa satırın tüm hücrelerini ekle.
3. **Sütun taraması**: her **x** için **IsColumnFull** doğruysa ekle (kesişimler zaten kümede).
4. **Puanlama**: **scoreGain += board.GetValue(pos)** (temizlemeden **ÖNCE** — sonra 0 dönerdi).
5. Küme listeye çevrilir, her hücre **Clear** edilir.
6. **new LineClearResult(cleared, scoreGain)**.

IsRowFull(board, y) / IsColumnFull(board, x) — herhangi biri boşsa **false**.

Rol: **ApplyMove**'un üçüncü ve son aşaması. **Puanın tek kaynağı**. Kesişim-tek-sayım adil ve deterministik puanlamayı garanti eder.

PlacementService — MERKEZİ PIPELINE (tek doğruluk kaynağı)

`public sealed class PlacementService` — *"The single central move-resolution pipeline.*

`ApplyMove` is

the one source of truth used by preview (scratch board), real moves (real board) and opponent evaluation."

Alanlar: `MergeResolver _mergeResolver`, `LineClearResolver _lineClearResolver` (kurucudan enjekte — orkestrasyon).

Metodlar:

- `CanPlace(board, piece, anchor)` — her hücre için `position = anchor + cell.Offset`; sınır dışı VEYA dolu ise `false`. Parça, taşmadan ve üst üste binmeden sığmalı.
- `ApplyMove(board, piece, anchor)` — **TEK DOĞRULUK KAYNAĞI:**
 1. **Doğrulama:** `if (!CanPlace(...)) return MoveResult.Invalid();` — geçersizde tahtaya hiç dokunulmaz (atomiklik).
 2. **Yerleştir (Aşama 1):** her hücre `board.SetValue(position, cell.Value)`, `placedCells`'e eklenir.
 3. **Birleştir (Aşama 2):** `_mergeResolver.Resolve(board, placedCells)` — zincir çözülür.
 4. **Temizle (Aşama 3):** `_lineClearResolver.Resolve(board)` — dolu satır/sütun temizlenir, puan.
 5. **Sonuç:** `new MoveResult(true, placedCells, mergeSteps, ..., scoreGain)`.
 - Aynı metod öniizlemede (scratch kopya), gerçek hamlede ve AI değerlendirmesinde çalışır → öniizleme sonucu gerçek hamleyle **matematiksel olarak birebir** aynı. Kural hiçbir yerde kopyalanmaz.
- `HasAnyValidMove(board, pieces)` — herhangi bir parça herhangi bir konuma sığıyor mu? Her parça × her hücre için `CanPlace`; ilk `true`'da kısa devre. Hiç yoksa `false` (oyun bitti / yenileme gerekli). Girdi genelde `TrayModel.Pieces`.

Rol: Gameplay'in orkestra şefi. Düşük seviye kural içermez; `MergeResolver` + `LineClearResolver`'ı doğru sırayla ve doğru tahtada çağırır. `ApplyMove`'un tek doğruluk kaynağı olması, öniizleme/gerçek/AI tutarlılığını **mimari düzeyde** garanti eder.

Pipeline Özeti — Uçtan Uca

1. **Üretim:** `PieceFactory.CreateRandomPiece()` → `TrayModel.SetPieces`.
2. **Öniizleme:** `BoardModel.CopyFrom` ile kopya + `ApplyMove` → sonuç öniizlenir (gerçek tahta bozulmaz).
3. **Uygulama:** `ApplyMove` gerçek tahtada: Yerleştir → Birleştir (BFS+toplama+zincir) → Temizle (HashSet tek-sayım).
4. **Dağıtım:** `MoveResult.ScoreGain` → `ScoreService.Add`; parça `TrayModel.Remove`.
5. **Fail-state:** `HasAnyValidMove(board, tray.Pieces)` false → oyun kaybedildi/yenileme.

7. App Katmanı (Kompozisyon Kökü) ve Input

`GameSessionController` uygulamanın **kompozisyon kökü** (composition root), diğer beş sınıf onun sahiplendiği yardımcılardır. `PieceDragController` ise Input katmanının tek sınıfıdır.

GameSessionController — kompozisyon kökü ve orkestratör

`public sealed class GameSessionController : MonoBehaviour` — sunum, input ve rakip katmanlarını

tanıtmaya izin verilen **tek yer**. Saf servisler UI'dan bağımsız kalır; bu sınıf hepsini bağlar.

Config alanları ([SerializeField]): `boardConfig`, `pieceSetConfig`, `failBannerSeconds = 2f`.

View'lar: `gameplayRoot`, `boardView`, `trayView`, `animationController`, `localScoreView`, `versusLocalScoreView` ("YOU"), `opponentScoreView`, `bestScoreLabel`, `bestScoreRoot`, `versusHudRoot`, `tutorialOverlay`, `gameOverPanel`, `mainMenuView`, `matchmakingOverlay`, `settingsGearButton`.

Controller'lar: `tutorialController`, `turnController`, `settingsPanel`, `sfxPlayer`, `musicPlayer`.

Runtime alanları: `BoardModel _board`, `BoardModel _previewScratch`, `PlacementService _placement`, `PieceFactory _factory`, `ScoreService _localScore/_opponentScore`, `GameSettings _settings`, `GameState _state = MainMenu`, `GameMode _mode`, `bool _tutorialOnly`, `SessionHud _hud`, `GameOverSequence _gameOver`, `InputGate _inputGate`, `PlayerProgress _progress`, `TrayModel _trayModel`.

Olaylar: `event Action<PlayerSide, MoveResult> MoveResolved`, `event Action<PlayerSide> MoveStarted`.

Property'ler: `Board`, `Placement`, `LocalScore`, `OpponentScore`, `IsResolving`,

`CanAcceptPlayerInput => _state == PlayerTurn && !IsResolving && !_inputGate.LockedBySettings` (kritik girdi kapısı).

Metodlar:

- `Awake()` — Tüm bağımlılıkları kurar: `GameSettings`, iki `BoardModel`, `PlacementService`(`new MergeResolver()`, `new LineClearResolver()`), `PieceFactory`, iki `ScoreService`, `PlayerProgress` (`+Load()`), `SessionHud`, `GameOverSequence`. Skor aboneliği, ses bağlama, tahta `Build`, tepsi `Initialize`, tutorial, `WireUi()`.
- `Start()` — Sadece `ReturnToMenu()` (oyun ana menüden başlar).
- `SetState(state)` — `_state`'in atandığı **tek yer**; gear butonu etkileşimini senkronize eder. Ayarlar maçı asla duraklatmaz, sadece yerel girdiyi kilitler.
- `WireUi()` — Menü, oyun-sonu, ayarlar ve tutorial olaylarını metodlara bağlar.
- Mod girişleri: `StartSolo()/StartTutorialOnly()` → `StartSoloFlow(tutorialOnly)`; `BeginSoloRun()` (kilit aç, tahta/skor sıfırla, tepsi doldur, `PlayerTurn`).
- `StartVsAI()` → `ConnectAndStartVsAI()` (coroutine): önce `turnController.Stop()`, sahte "rakip bulunuyor" overlay, mod VsAI, HUD versus, tepsi doldur, `turnController.Initialize +`

StartTurns.

- **Hamle API'si:** `PreviewMove(...)` (scratch'te dener), `CanSubmitMove(...)` (üç kapı: `CanAcceptPlayerInput` + `CanPlace` + tutorial izni), `ApplyPlayerMove/ApplyOpponentMove` → `ApplyMoveInternal(...)`.
- `ApplyMoveInternal(piece, anchor, side, score)` — Otoriter tahtaya uygular; geçersizse çıkar. Geçerliyse parçayı model VE view'dan **birlikte** tüketir (`ConsumePiece` + `_trayModel.Remove`), `Debug.Assert` ile senkron kontrolü, `MoveStarted`, skor, `ResolveRoutine`. Çağırınlar asla ön-tüketmez → yarım tüketim olamaz.
- `ResolveRoutine(result, side)` (coroutine) — `ResolvingMove`, tahta yenile, ses, animasyon, tepsi doldur, `OnMoveResolved`.
- `OnMoveResolved(side, result)` — Solo: tutorial bildirimi / `SoloGameOver` / `PlayerTurn`. VsAI: `MoveResolved` olayı (`TurnController` dinler).
- Oyun sonu: `SoloGameOver()/EndVsGame()` → `_gameOver.PlaySolo/PlayVersus`.
- Tur/input: `EnterPlayerTurn()`, `EnterOpponentTurn()`, `SetInputLocked(bool)` (state'e dokunmadan), `HasAnyValidMove()`, `GetTrayPieces()`, `ApplyTimeoutPenalty(side)`.
- Aktif sürüklenme: `SetActiveDrag`, `ClearActiveDrag`, `CancelActiveDrag`, `NotifyInvalidDrop` (hepsi `_inputGate`'e delege).
- `ApplyTutorialStep(step)`, `OnSettingsOpened/Closed` (VsAI'da timer duraklaması YOK — gerçek online maç gibi), `OpenInGameSettings()`, `Replay()`, `ReturnToMenu()` (savunmacı: `CancelActiveDrag` + `_inputGate.Unlock()`), `RefillTray()`, `OnLocalScoreChanged`.

Rol: Uygulamanın kompozisyon kökü. `Awake()`'te tüm bağımlılıkları tek noktada `new`'ler, `WireUi()`'de tüm olayları bağlar. Yardımcı sınıfların çoğu `MonoBehaviour` değil → sahne sözleşmesi etkilenmez.

SessionHud

`internal sealed class SessionHud` — `MonoBehaviour` **değil**, düz sınıf ("scene contract is untouched").

Tüm alanları `readonly`, kurucudan atanır: `_gameplayRoot`, `_mainMenu`, `_bestScoreRoot`, `_versusHudRoot`, `_localScoreView`, `_versusLocalScoreView`, `_opponentScoreView`, `_bestScoreLabel`.

Metodlar: `ShowGameplay()/HideGameplay()`, `EnterSolo(localScore)` (versus HUD gizle, en iyi skor göster,

merkezi skoru `Bind`), `EnterVersus(local, opponent)` (versus HUD göster, "YOU"/"OPPONENT" pilleri bağla,

merkezi büyük skoru gizle), `RestoreCentralScore()`, `SetBestScore(value)`.

Rol: Saf view geçişi + skor bağlama katmanı; oyun kuralı içermez. Orkestratörü inceltir.

InputGate

`internal sealed class InputGate` — `MonoBehaviour` değil. Alanlar: `PieceDragController` `_activeDrag`,

`bool LockedBySettings { get; private set; }.`

Metodlar: `SetLocked(bool)` (kilitliyse `CancelActiveDrag`), `Unlock()` (savunmacı temizlik), `SetActiveDrag/ClearActiveDrag` (yalnız aynı referanssa), `CancelActiveDrag()`.

Rol: İki şeyi izler: girdi kilitli mi (Settings açık) ve hangi drag canlı. **GameState'ten kasıtlı ayrı** — kilitleme gerçek oyun/tur durumunu asla bozmaz. `CanAcceptPlayerInput`'un üçüncü koşulu buradan gelir.

GameOverSequence

`internal sealed class GameOverSequence` — `MonoBehaviour` değil. Alanlar (readonly): `_overlay`, `_animation`, `_boardView`, `_sfx`, `_panel`, `_config`, `_bannerSeconds`.

Metodlar (coroutine): `PlaySolo(localScore)` → `ShowFailBanner` → `PlayWave` → `_panel.ShowSolo(...)`

(skor panel-gösterim anında okunur, erken snapshot yok); `PlayVersus(...)` aynı akış; `ShowFailBanner()`

(oyun sonu sesi + "No valid moves left" pankartı); `PlayWave()` (tahta solma dalgası).

Rol: Oyun-sonu sunum koreografisi: ses → pankart → solma dalgası → sonuç paneli. Oyunun ne zaman bittiğine session, nasıl gösterileceğine bu sınıf karar verir.

PlayerProgress

`public sealed class PlayerProgress` — `MonoBehaviour` değil; `PlayerPrefs`'i sarmalayan **tek yer**.

Alanlar: `_bestScoreKey`, `_tutorialSeenKey`, `int BestScore { get; private set; }`, `bool HasSeenTutorial { get; private set; }`.

Metodlar: `PlayerProgress(string keyPrefix = "")` (testler izole prefix geçebilir), `Load()`, `MarkTutorialSeen()`, `TrySubmitBestScore(score, mode, tutorialActive)` — saf koruma:

```
if (mode != GameMode.Solo || tutorialActive || score <= BestScore) return false;
```

Yalnız gerçek solo skoru mevcut en iyiden kesinlikle büyükse kaydeder.

Rol: Kalıcı oyuncu meta verisi. Best-score guard'ı saf ve test edilebilir bir metod olarak izole.

PieceDragController — fare + dokunma tek yol

`public sealed class PieceDragController : MonoBehaviour, IPointerDownHandler, IDragHandler, IPointerUpHandler`

— bu grupta `MonoBehaviour` olan tek yardımcı; üç UGUI pointer arayüzünü uygular.

Inspector: `pickupScale = 1.1f`, `dragYOffset = 45f` (parmağın üstünde dursun; yarım hücreden az → anchor değişmez), `returnSpeed = 18f`.

Private: `_session`, `_boardView`, `_pieceView`, `_homeSlot`, `_rect`, `_canvas`; durum: `_dragging`, `_lastAnchor`, `_hasAnchor`, `_grabOffset`, `_previewAnchor`, `_previewShown` (GC'siz — yalnız hücre değişince yeniden çizer), `Coroutine _snapRoutine`.

Metodlar:

- `Initialize(session, boardView, pieceView, homeSlot)`.
- `OnPointerDown` — `!CanAcceptPlayerInput` ise çık; `_dragging = true`, `_hasAnchor = false` (kritik: eski anchor'ın sürüklemesiz TAP'te bayat hücreye hamle yapmasını önler), `SetActiveDrag`, canvas'a taşı, `pickupScale`, raycast kapat, `_grabOffset` hesapla.
- `OnDrag` — parçayı `world + _grabOffset + dragYOffset`'e taşı; `PiecePlacementPreview.Tick(...)` (AI ile aynı paylaşılan adım); yalnız hücre değişince yeniden çizer.
- `OnPointerUp` — `ClearActiveDrag`, önlizeme temizle; `_hasAnchor && CanSubmitMove` ise `ApplyPlayerMove` (session model+view'ı birlikte tüketir); değilse geçersiz ses + `ReturnToSlot`.
- `CancelDrag()` — timeout gibi durumlarda **anında** (tween'siz) slot'a döndürür (rakip parçayı hemen alacak).
- `ReturnToSlot(animate)`, `SnapHome()` (Lerp tween), `ScreenToWorld(eventData)`.

Rol: Tek tepsi parçasının pointer sürüklemesi. **Fare + dokunmanın tek birleşik yolu** UGUI pointer arayüzlerinden gelir (Unity ikisini de aynı `PointerEventData`'ya normalize eder). Down → Drag → Up akışı; hamle **paylaşılan pipeline** (`ApplyPlayerMove`) üzerinden gönderilir. Ayar kilidi/timeout `CancelDrag()` ile anında sonlandırılabilir. İnsan ve AI için tek doğrulama yolu.

8. Opponent Katmanı (AI Rakip Mimarisi)

Bu klasör, "Vs AI" modunun tüm yapay zekâ mantığını içerir. Mimarinin dört sütunu vardır:

1. **Karar** → `OpponentMoveEvaluator` + `OpponentMove` — ne oynanacak?
2. **Koreografi/Planlama** → `OpponentActPlanner` + `OpponentBeat` — hamle nasıl "insan gösterisi" olarak sahnelenecek?
3. **Sunum** → `OpponentPresentationController` — plan ekranda nasıl canlandırılacak?
4. **Döngü/Zaman** → `TurnController` + `TurnTimer` — sıralar, 20 sn sayaç, timeout cezası.

Temel felsefe: rakip ne mükemmel ne rastgele oynar; oyuncuyla ****birebir aynı**

`PlacementService.ApplyMove`

pipeline'ını **kullanır; skoru yakın tutmak için** rubber-band** uygular; "sahte oyalanma" yapmaz —

ekrandaki tereddüt gerçek karar belirsizliğini yansıtır (blöf değil, **dürüst gösteri**).

OpponentMove (immutable DTO)

Bir aday/seçilmiş hamleyi taşır: `PieceInstance` `Piece`, `Vector2Int` `Anchor`, `float` `Score` (heuristic değeri). `Score`, sıralama ve "top-gap" (en iyi-ikinci farkı) için kritiktir.

OpponentBeat (enum + readonly struct)

Rakip turunun "tek bir anını" temsil eden saf veri — kararı hareketten ayırır (plan mantığı **unit-test edilebilir**).

`OpponentBeatType`: `SelectDelay` (düşün, parçayı al), `TravelTo` (hücreye git), `Hover` (beklet), `Misdrop` (geçersiz hücreye git, kırmızı glow, geri sek), `SwapFakeout` (farklı parça al, düşün, geri koy), `FinalPlace` (gerçek çapaya bırak). Struct alanları: `Type`, `Cell`, `Seconds`, `Piece` (yalnız swap için).

Private kurucu + 6 statik fabrika (her beat türü yalnız anlamlı alanları doldurur). Bir tur = `List<OpponentBeat>`.

OpponentMoveEvaluator — KARAR MOTORU

`public sealed class` (saf mantık, test edilebilir). İnandırıcı hamle seçer: her aday ****oyuncunun aynı**

`ApplyMove`'uyla** scratch tahtada simüle edilir, heuristic'e çevrilir, en iyi N aday arasından **ağırlıklı rastgele** seçilir. N, skor farkına göre değişir → **rubber-band**.

Sabit ağırlıklar: `ScoreGainWeight=10`, `MergeStepWeight=5`, `ClearedCellWeight=3`, `MergedValueWeight=1`,

`NoEffectPenalty=4`, `TopWeights={3,2,1}`. `ObviousMoveGap=25`: en iyi aday ikinciye bu kadar geçerse hamle "aşikâr" sayılır, rubber-band devre dışı — insan böyle hamleyi kaçırmaz.

Alanlar: `System.Random _random` (seed'lenebilir), `int _rubberBandThreshold` (varsayılan 15), `float LastTopGap` (public — sunum kararsızlıkta daha uzun tereddüt etsin diye okur).

Metodlar:

- `ChooseMove(board, placement, pieces, scoreDelta)` — Ana giriş. `scoreDelta` = rakip – yerel skor.
- 1. Her tepsi parçası × her hücre için `CanPlace` kontrolü.
- 2. Yerleştirilebiliyorsa `scratch.CopyFrom(board)` + `ApplyMove` ile simüle, `Heuristic` ile puanla, listeye ekle.
- 3. Aday yoksa `LastTopGap=0, null` (pas/oyun sonu).
- 4. Puana göre azalan sırala; `LastTopGap` = en iyi – ikinci.
- 5. Genişlik: `LastTopGap >= 25` ise **1**, değilse `SelectionWidth(scoreDelta)`.
- 6. `WeightedPickFromTop` ile seç.
- `SelectionWidth(scoreDelta)` — **rubber-band özü:** önde (`> threshold`) → **3** (alt-optimal insani hamle şansı, oyuncu yakalar); geride (`< -threshold`) → **1** (tek en iyi hamle, baskı); yakın → **2**.
- `WeightedPickFromTop(sorted, maxTop)` — `TopWeights={3,2,1}` → 1. aday 3/6, 2. 2/6, 3. 1/6 olasılık; `_random.Next(totalWeight)` ile seçer.
- `Heuristic(result)` — `ScoreGain10 + MergeSteps.Count5 + ClearedCells.Count3 + mergedValue1`; ne merge ne clear varsa **-4** ceza. Verimli ama insani hamleleri ödüllendirir.

OpponentActPlanner — KOREOGRAFİ PLANLAYICISI

`public sealed class` (seed'lenebilir RNG, test edilebilir). Seçilmiş hamleyi **sınırlı, insan-okur** beat listesine çevirir: düşün → 1-4 decoy deneme → ara sıra misdrop → tereddüt → bırak. İlke: **dürüst gösteri** —

hiçbir decoy, gerçek hamleden **daha iyi** görünen bir hücreyi göstermez.

Alanlar: `_minAttempts/_maxAttempts` (decoy bandı), `_preMoveDelayMin/Max`, `_hoverHesitation`, `_misdropChance`, `_swapFakeoutChance`, `_rng`, `_cellScratch` (yeniden kullanılan buffer), `_simBoard`.

`Plan(board, placement, move, thinkScale, trayPieces)` — **ana metod:**

1. `_simBoard.CopyFrom(board) + chosen = ApplyMove(...)` (dürüst-decoy referansı).
2. **Kararsızlık:** `torn = InverseLerp(0.7, 1.15, thinkScale)` — `thinkScale<1` → net hamle → `torn≈0` (düz git); `≈1.15` → yakın karar → `torn≈1` (adaylar üzerinde düşün).
3. **Ortak fake bütçesi:** swap+decoy+misdrop toplamı ≤ 3 , nadiren (%5) 4.
4. **Swap-fakeout:** yalnız gerçekten kararsızsa (`torn>0.5`) — `PickOtherPiece` + `FindHonestCellFor`.
5. `SelectDelay` beat: `RangeF(preMoveDelayMin, Max) * thinkScale`.
6. **Decoy sayısı** kararsızlıkla ölçekli: aşikâr → 0; `torn>0.75` → en az 1. `CollectTryCells` ile toplanır.
7. Her decoy: `TravelTo` + karakterize `Hover` (%40 hızlı bakış, %45 normal, %15 uzun düşünme; kısa ölü zaman; %30 ekstra linger).
8. **Misdrop:** bütçe/zar tutarsa `FindInvalidCell` ile geçersiz hücre.
9. **Final:** çapaya `TravelTo` + son `Hover` (`thinkScale` ölçekli) + `FinalPlace`.

Yardımcılar: `CollectTryCells` (dürüst decoy toplayıcı — gerçek hamlede olmayan clear/merge üreten hücreyi

atlar, sonra kısmi Fisher-Yates karıştırır), `PickOtherPiece`, `FindHonestCellFor`, `FindInvalidCell`.

OpponentPresentationController — SUNUM/HAREKET MOTORU

`public sealed class : MonoBehaviour`. Rakibin hamlesini **görünür** kılar. Hareket eased ve hafif kavisli;

hover'lar nazikçe salınır; zamanlama/yol jitter'lı → **insan eli** gibi okunur. Rakip asla ekran dışı oynamaz.

Planner ne oynanacağına karar verir; controller sadece oynatır.

Serialize tuning: `moveSpeed=1000`, `dragYOffset=45`, `hoverHesitation=0.5`,
`preMoveDelayMin/Max=1.6/2.7`,

`bobAmount=7`, `pathCurve=0.13`, `minAttempts=0`, `maxAttempts=2`, `misdropChance=0.05`,
`swapFakeoutChance=0.25`.

Metodlar:

- `Awake()` → `RebuildPlanner()` (planner'ı tuning alanlarından kurar).
- `TickPreview()` → oyuncuyla **aynı** `PiecePlacementPreview.Tick` (glow her karede parçayı takip eder).
- `PlayMove(move, tray, board, session, thinkScale)` (coroutine) — Parça view'ını bul; canvas/layer/kamera hesabı (GameplayRoot içinde render → menüde yüzmez);
`_planner.Plan(...)` → beat listesi; her beat'i switch ile oynat
(`SwapFakeout/SelectDelay/TravelTo/Hover/Misdrop/FinalPlace`). `FinalPlace` → `session.ApplyOpponentMove` (model+view birlikte).
- `SwapFakeout(...)`, `AttemptMisdrop(...)` (aynı SFX + `NotifyInvalidDrop`), `CellTargetPos(...)` (referans hücreyi hedefe getirir → glow/parça/yerleşim aynı hücrede).
- **İdatif el-hareketi:** ayrı cursor yok — hareket parçanın transform'una eklenir; sürekli ilerleyen `_idleClock` asla sıfırlanmaz (el hep "canlı"). `RerollIdle()` her dinlenmede genlik/ritim/nefes değerlerini yeniler; `IdleOffset(scale)` iki sinüs oktavı + breathe + drift ile organik salınım üretir; hızlı harekette söner.
- `MoveTo(...)` — eased, kavisli (bowed mid-point Bézier) + hız-dalgacığı (ripple, envelope uçlarda 0 → temiz başla-dur). `Hover(...)` — yumuşak nefes-salınımı.

TurnTimer — 20 SANİYE SAYAÇ

`public sealed class : MonoBehaviour`. Oyuncu turu için geri sayım; **kasıtlı olarak duraklatılamaz**

(gerçek online maç gibi — ayarlar paneli bile durduramaz).

Alanlar: `_remaining`, `_total`, `_running`. **Event'ler:** `Ticked(remaining, total)`, `TimedOut`.

Metodlar: `StartCountdown(duration)` (hemen bir `Ticked` yayınlar), `Stop()`, `Update()` (`_remaining -= Time.deltaTime`;

`≤0` → 0'a sabitle, dur, `Ticked(0, total)` + `TimedOut`; değilse `Ticked(remaining, total)`).

TurnController — DÖNGÜ VE ORKESTRASYON

`public sealed class : MonoBehaviour`. Tur sırasını, 20 sn sayacı, timeout cezasını ve rakip turunu yönetir.

Çekirdek oyun bunu asla referans almaz — session küçük bir API sunar (tek yönlü bağımlılık). Tüm maç,

event ping-pong yerine **tek okunabilir yukarıdan-aşağı coroutine** (`MatchLoop`) olarak yazılmıştır.

Serialize: `turnTimer, opponentPresentation, turnView, timerView`; tuning `playerTurnSeconds=20, rubberBandThreshold=15`. **Runtime:** `_session, _tray, _boardView, _evaluator`, bayraklar `_moveResolved/_timedOut/_moveStarted, CurrentTurn` (PlayerSide).

Metodlar:

- `Initialize(session, tray, boardView)` — `_evaluator` kur; event abonelikleri (`MoveResolved, MoveStarted, Ticked, TimedOut`).
- `StartTurns()` → `MatchLoop` coroutine. `Stop()` → deaktive + `StopAllCoroutines()` (devam eden sunumu öldür).
- `MatchLoop()` — **MAÇ DÖNGÜSÜ:**
 - **A) Oyuncu turu:** bayrakları sıfırla, `BeginPlayerTurn()`, `WaitUntil(_moveResolved || _timedOut || !_active)`.
 - **Gerçek timeout** (`_timedOut && !_moveStarted`): `CancelActiveDrag`, `ApplyTimeoutPenalty(Local)`, `turnTimer.Stop`, `turnView.ShowTimeout()` (flash → %5 görünür), 1 sn bekle.
 - **Aynı-kare çıkışması** (`!_moveResolved`): hamle timer'la aynı anda commit edilebilir → **hamle kazanır** (ceza yok), resolve'unu bekle.
 - Fail-state: `!HasAnyValidMove()` → `EndGame()`.
 - **B) Rakip turu:** `BeginOpponentTurn()`, `scoreDelta = OpponentScore - LocalScore`, `move = ChooseMove(...)` (null → `EndGame`). `thinkScale = Lerp(1.15, 0.6, InverseLerp(0, 25, LastTopGap))` (yakın karar → uzun tereddüt). `PlayMove(...)`, resolve bekle, tekrar fail-state.
- `BeginPlayerTurn()` / `BeginOpponentTurn()` — her ikisi de 20 sn sayaç çalıştırır (rakibinki turuncu tint, ceza tetiklemez). `OnMoveStarted(side)` — commit anında sayacı dondur; Local ise `_moveStarted=true` (aynı-kare timeout'u yener). `OnMoveResolved(side, result)` — **yalnız sırası olan taraf** beklemeyi kapatır (geç gelen resolve bir sonraki turu atlatmasın). `OnTimeout()` — Local sırasındaysa `_timedOut=true`. `EndGame()` → `EndVsGame()`.

Turn Döngüsü — Özet Akış

```
MatchLoop
├─ BeginPlayerTurn → TurnTimer(20s)
│   └─ hamle → OnMoveStarted(dur) → OnMoveResolved → _moveResolved
│       └─ süre bitti → OnTimeout → _timedOut → CancelActiveDrag + ceza + ShowTimeout
├─ HasAnyValidMove? değilse → EndGame
├─ BeginOpponentTurn → TurnTimer(20s, turuncu, cezasız)
│   └─ Evaluator.ChooseMove(scoreDelta) → OpponentMove + LastTopGap
│       └─ thinkScale = f(LastTopGap)
│           └─ Presentation.PlayMove → Planner.Plan → beat'ler (dürüst decoy/swap/misdrop) → FinalPlace →
ApplyOpponentMove
└─ HasAnyValidMove? değilse → EndGame
```


9. Presentation Katmanı (Görseller + Animasyonlar)

Genel ilke: ****modelden gelen saf veri (BoardModel, MoveResult, PieceInstance) burada görselleştirilir;**

oyun mantığı burada tutulmaz. **** Kural/matematik Core/Gameplay/App'te yaşar; bu katman onları çizer.**

BoardView

public sealed class : MonoBehaviour. UGUI oyun tahtası ızgarasını inşa eder, ****ekran koordinatlarını**

ızgara anchor'larına çevirir **** (koordinat matematiği burada; ayrı "input-mapper" yok) ve sürükleme**

önizleme vurgularını yönetir.

Alanlar: **RectTransform cellContainer, CellView cellPrefab, BoardConfig _config, CellView[] _cells** (1D, erişim $y*_width+x$), $width/height/_cellSize$, **List<Vector2Int> _highlighted**

(önizleme vurguları), **_tutorialCells** (kalıcı tutorial hedefleri).

Metodlar:

- **Build(config)** — ızgarayı sıfırdan kurar; konteyner boyutu, her (x,y) için **cellPrefab** klonla, merkez $(x+0.5)*_cellSize$.
- **Refresh(board)** — Model değerlerini görsele senkronlar; önce **ResetVisual()** (game-over dalgasının bıraktığı görünmez hücreleri replay'de düzelt), sonra **SetEmpty/SetValue(value, GetColorForValue(value))**.
- **GetCell(pos), GetCellWorldCenter(pos)**.
- **TryScreenPointToAnchor(screenPos, cam, out anchor)** — kritik dönüşüm: **ScreenPointToLocalPointInRectangle**, pivot'tan bağımsız **rect.xMin/yMin** kaydırma, **FloorToInt(relX/_cellSize)**, **margin=1f** tolerans. Geçerliliği burası değil **CanPlace** belirler.
- **ShowMovePreview(result)** — **PlacedCells→ValidPlacement, MergeStep.SourceCells→MergeSource, ClearedCells→LineClear**.
- **ShowInvalidPreview(cells), ClearPreview()** (sonra **ApplyTutorialHighlight()** — tutorial ışığı kalıcı).
- Tutorial vurgu metodları, **Highlight(pos, type)**.

Rol: Model verisini piksellere çeviren ana köprü ve tüm ekran→ızgara input matematiğinin sahibi.

CellView

public sealed class : MonoBehaviour. Tek tahta hücresi — **yalnız görsel durum, oyun mantığı yok;**

küçük tween'ler (pop, fade) animasyon controller'ının sıralaması için dışa açılır.

Alanlar: `Image background/highlightOverlay`, `TMP_Text valueText`, `CanvasGroup canvasGroup`,

`Sprite filledSprite/emptySprite`, vurgu renkleri
(`validColor/invalidColor/mergeColor/lineClearColor`),

`_emptyColor`, `Coroutine _popRoutine`. **Property:** `RectTransform`, `FilledSprite` (uçan hayalet bloklar aynı sprite'ı kullansın).

Metodlar: `Configure(emptyColor)`, `SetEmpty()`, `SetValue(value, color)`, `SetHighlight(type)`,

`ClearHighlight()`, `ResetVisual()` (scale=1, alpha=1), `PlayPop()` (Punch 1.18/0.16), `PlayPunch()` (1.34/0.22, merge landed),

`FadeOut(duration)` (game-over dalgası), `PopRoutine` (iki fazlı ölçek).

Rol: En küçük görsel birim; tüm "juice" yapı taşları burada, koreografi `GameplayAnimationController`'da.

GameplayAnimationController

`public sealed class : MonoBehaviour`. Çözülen hamlenin tüm "juice"unu oynatır: yerleşim pop'ları, animasyonlu merge (soğurulan bloklar büyüüp sabit hedefe uçar, hedef toplamı punch+glow ile açar),

satır temizleme parlaması, uçan skor, game-over solma dalgası. Girdi: `MoveResult`.

Alanlar: `RectTransform floatingScoreLayer`, `TMP_Text floatingScorePrefab`, `Sprite glowSprite`,

`BoardConfig boardConfig`, zamanlamalar (`mergeEffectTime=0.09`, `lineClearFlashTime=0.28`, `cellFadeTime=0.18`,

`gameOverWaveStep=0.02`), `Stack<GameObject> _glowPool` (GC azaltır).

Metodlar:

- `PlayMove(result, board)` (coroutine) — Tahta zaten son duruma refresh olduğundan, merge katılımcılarını `OriginalValue` ile **pre-merge** haline geri boyar (blok yok olup görünmesin). Sonra `PlacedCells` pop, her `MergeStep` için `PlayMergeStep` (zincir tek tek), `HasLineClear` → vurgu+glow+uçan skor+bekleme+`SetEmpty`.
- `PlayMergeStep(step, board)` — hedef sabit; kaynaklar `SetEmpty`, yerlerine uçan hayalet bloklar; `mergeEffectTime` boyunca büyü/otur + `SmoothStep` hareket; sonda hedef `ResultValue` açar + `PlayPunch` + `SpawnGlow`.
- `SpawnGhostBlock(...)`, `ColorFor(value)`, `PlayGameOverWave(...)` (üstten alta dalga), `ShowFloatingScore(...)`, `FloatAndFade(...)`, glow havuzu (`SpawnGlow/GetGlow/GlowRoutine`), `CenterOf(cells, board)`.

Rol: Soyut `MoveResult`'u zamanda koreografiye çeviren "juice motoru"; coroutine'ler animasyonu oyun akışıyla senkronlar.

PiecePlacementPreview — ORTAK YERLEŞTİRME ÖNİZLEMESİ

`public static class` — **MonoBehaviour değil, statik yardımcı**. Mimari açıdan kritik: bir parçanın canlı

ekran konumunu tahta anchor'una ve hücre vurgusuna çeviren **tek doğruluk kaynağı**. Hem oyuncu sürüklemesi

hem AI rakip buradan geçer — glow hep parçanın kendi konumundan sürülür, moda özel kod yolu yok.

Metodlar (static):

- `TryResolveAnchor(piece, board, cam, out anchor)` — `piece.ReferenceCellWorldCenter` → ekran → `TryScreenPointToAnchor` → `anchor = refCell - piece.ReferenceOffset`. Footprint hep parçanın tam altında (lift/grab/idle'dan bağımsız).
- `Show(board, session, piece, anchor)` — `session.PreviewMove` sonucu; geçerliyse `ShowMovePreview`, değilse `ShowInvalidPreview(Footprint(...))`.
- `Tick(...)` — her karede çalışan tek önizleme adımı; anchor değişince yeniden çizer, tahta dışında `ClearPreview`. Hem drag controller hem AI aynı `Tick`'i çağırır.
- `Footprint(piece, anchor)`.

Rol: Oyuncu ve AI için tek birleşik önizleme; kod tekrarını ve mod-farklı hataları önler.

PieceCellView / PieceView / PieceTrayView

PieceCellView (`MonoBehaviour`) — bir tepsi/sürükleme parçasının tek görsel hücresi.

`SetValue(value, color)`

ile sprite+renk+metin. `CellView`'in parça muadili ama highlight/animasyon yok.

PieceView (`MonoBehaviour`) — bir `PieceInstance`'in görsel karşılığı; her hücreyi tahtayla **aynı hücre

boyutunda** dizer (sürükleme hizalaması eşleşir). Property'ler: `Piece`, `HasReferenceCell`, `ReferenceOffset`,

`ReferenceCellWorldCenter`. `Build(piece, config)` — bounding box hesaplar, parçayı slotta ortalar, hücreleri

`(offset+0.5)*cellSize` konumlandırır, ilk hücreyi **referans hücre** kaydeder (anchor eşleme için).

`SetScale(scale)`, `SetRaycastTarget(bool)`.

PieceTrayView (`MonoBehaviour`) — canlı tepsi parçalarını ve sürükleme controller'larını tutar. Part 2'de

tepsi **paylaşımlı** (oyuncu ve rakip aynı parçalardan seçer). `const TrayScale=0.72f`.

`ShowPieces(pieces)` —

slotlara **ortalayarak** yerleştirir (tek tutorial parçası ortada), her biri için `PieceView` + `PieceDragController`

kurar. `ConsumePiece(view)`, `FindViewFor(piece)`, `ActiveCount` (session otoriter `TrayModel` ile senkron doğrular).

10. UI Katmanı (Menü, Skorboard, Tur/Sayaç, Oyun Sonu)

UI sınıfları iş mantığı taşımaz; buton tıklamalarını **Action** event'leriyle koordinatöre (App) iletir ve model verisini metne/çubuğa çevirir.

MainMenuView

MonoBehaviour. Minimal ana menü: Play (solo), Play vs AI, Tutorial, ayar dişlisi (kasıtlı store/reklam yok).

Event'ler: `PlaySolo`, `PlayVsAi`, `PlayTutorial`, `OpenSettings`. `Awake()` her butona null-güvenli listener bağlar (ilgili event'i tetikler). `Show()/Hide()` → `root.SetActive`.

ScoreView

MonoBehaviour. Bir `ScoreService`'e bağlanıp skoru gösterir. `Bind(score, labelText)` — önceki aboneliği

kaldırır (çift abonelik önlenir), yeni servise abone olur, mevcut skoru hemen gösterir.

`OnScoreChanged(value)`

metni günceller. `OnDestroy()` aboneliği temizler (sızıntı önlemi). Vs AI'da iki ayrı ScoreView farklı servislere bağlanır.

TurnView

MonoBehaviour. Sıranın kimde olduğunu gösterir. Metinler: `localText="Your Turn"`, `opponentText="Opponent's Turn"`,

`timeoutText="Time's up! -5%"`. `SetVisible(bool)`, `SetTurn(side)` (Local→"Your Turn"), `ShowTimeout()`

(zaman aşımı cezasını kısa süre gösterir).

TimerView

MonoBehaviour. Sıra süresini sayaç + dolum çubuğu ile gösterir. Renkler: `normalColor` (mavi/sizin), `opponentColor` (turuncu/rakip), `warningColor` (kırmızı/az kalınca). `SetOwner(side)` bar tonunu seçer.

`SetTime(remaining, total)` — `CeilToInt` ile saniye; **optimizasyon:** yalnız saniye değişince metni günceller

(`_lastShownSecond` cache, gereksiz string tahsisini önler). Dolum, sprite olmadığından `fillAmount` yerine

`fillBar.rectTransform.anchorMax.x = remaining/total` ile sürülür; oran < 0.3 → warning.

GameOverPanel

MonoBehaviour. Oyun sonu paneli. `Awake()` buton listener'larını bağlar (panel sahnede pasif başlar, kod açar).

`Configure(onReplay, onMainMenu)` callback enjekte eder. `ShowSolo(finalScore)` — versus grubunu kapatır,

"Score: X" + "Game Over". `ShowVersus(playerScore, opponentScore)` — iki skoru yazar, karşılaştırmayla

"You Win!/You Lose/Draw". `Show()/Hide()`.

MatchmakingOverlay

MonoBehaviour. Vs AI öncesi sahte "rakip bulunuyor" kaplaması — "sahte gerçek-zamanlı çok oyunculu"

illüzyonunu satar (**tamamen kozmetik, ağ yok, rakip lokal ve scriptli**).

`searchingSecondsMin/Max=2/2.8`,

`foundHoldSeconds=0.5`. `PlayConnecting()` (coroutine) — rastgele arama süresi boyunca noktaları animasyonlar,

"Opponent found!" yazıp bekler ve kapanır.

TutorialOverlay

MonoBehaviour. Mevcut tutorial talimatını ve geçici tahta banner'ını gösterir.

`ShowInstruction(text)`,

`HideInstruction()`, `ShowBanner(text, duration)` (coroutine — aç, bekle, kapat).

`TutorialController`'ın

metinsel geri bildirim yüzeyi; ayrıca "No valid moves left" başarısızlık banner'ı için kullanılır.

SafeArea

MonoBehaviour, `[RequireComponent(typeof(RectTransform))]`. Bu `RectTransform`'u cihazın güvenli alanına

sığdırır (çentik/yuvarlak köşe kaçınması), arkaplan tam ekran kalır. `Awake/OnEnable/Update` — güvenli alan

veya ekran boyutu değişince `Apply()` (yön/rotasyon için). `Apply()` güvenli alanı normalize edip anchor'ları ayarlar.

11. Tutorial ve Settings Katmanları

TutorialController (Tutorial)

MonoBehaviour. Üç zorunlu solo tutorial adımını (yerleştirme, merge, satır temizleme) yönetir.

Tahta

matematiğine **asla dokunmaz** — session'dan hazır (preset) adım yüklemesini ister ve izin verilen anchor'ları

kapıda kontrol eder; yanlış yerleştirmeler reddedilir, ilerlemez.

Alanlar: `TutorialStepDefinition[] steps`, `TutorialOverlay overlay`, `successHoldSeconds=1.8`, `PraiseMessages={"Nice!", "Great!", "Perfect!"}`, `_session`, `_index=-1`. **Property:** `IsRunning`, `HasSteps`.

Event: `Completed` (tüm adımlar bitince; session gerçek run başlatır).

Metodlar: `Initialize(session)`, `StartTutorial()` (adım yoksa `Completed`+çık; varsa `LoadCurrentStep`),

`StopTutorial()` (coroutine'leri durdur; move-gating normal oyuna sızmasın — `Completed` tetiklemez),

`IsMoveAllowed(piece, anchor)` (çalışmıyorsa/izinli anchor yoksa true; aksi halde anchor izinliyse true),

`NotifyMoveCompleted(result)` → `AdvanceAfterHold` (övgü mesajı, `successHoldSeconds` bekle, sonraki adım veya

`Completed`), `LoadCurrentStep()` (`session.ApplyTutorialStep(step)` + `overlay.ShowInstruction`).

Rol: Öğreticiyi scriptleyen ince kontrolör; tahta işini session'a delege eder, yalnız adım sırasını ve izinli hamleleri yönetir.

GameSettings (Settings)

`public sealed class` — `MonoBehaviour` **değil**, saf C#. Kaydetme yok; değerler her açılışta sıfırlanır (in-session).

Property'ler: `SfxEnabled`, `BgmEnabled`, `VibrationEnabled` (hepsi varsayılan true). **Event:** `Changed`.

Metodlar: `SetSfxEnabled/SetBgmEnabled/SetVibrationEnabled(bool)` — property'yi set edip `Changed?.Invoke()`.

Rol: Ayarların merkezi modeli; `MusicPlayer/SfxPlayer` dinler, `SettingsPanel` yazar (observer deseninin çekirdeği).

SettingsPanel (Settings)

MonoBehaviour. Açıldığı yere göre üç layout: **oyun içi** (Restart + Main Menu + Close), **oyun içi tutorial**

(Restart anlamsız → Main Menu + Close), **ana menüden** (yalnız toggle'lar + Close, kısa kart). Vs AI'da

açılınca sıra zamanlayıcı **duraklamaz** (Opened/Closed dinleyicileri yalnız yerel girdiyi kilitler).

Alanlar: `root`, `RectTransform card`, üç `Toggle` (sfx/bgm/vibration), üç `Button`, layout ayarları (kart yükseklikleri `fullHeight=1000/noRestartHeight=860/compactHeight=720`), `_settings`.

Event'ler: `Opened`, `Closed`, `Restart`, `MainMenu`.

Metodlar: `Awake()` (buton bağlama; Main Menu `root`'u kapatır ama `Closed` tetiklemeden — session run'ı

kendi söker, bu kritik regresyon düzeltmesiydi), `Bind(settings)` (toggle `isOn` = ayar; `onValueChanged` →

`settings.SetXEnabled`), `Show(allowRestart)/ShowFromMenu()`, `Open(inGame, allowRestart)` (buton görünürlüğü +

kart yüksekliği), `Hide()` (`root` kapat + `Closed` tetikle → zamanlayıcı devam).

MediaPlayer (Settings)

MonoBehaviour. Tek arkaplan müziğini döngüyle çalar, Music toggle'ına göre kapılanır. Eksik klip sorunsuz atlanır.

`Bind(settings)` — `Changed += Apply`, `audioSource` yapılandır (`loop=true`, `playOnAwake=false`, `volume=0.45`).

`OnDestroy()` aboneliği kaldırır. `Apply()` — `BgmEnabled` açıksa çal, kapalıysa `Pause`.

SfxPlayer (Settings)

MonoBehaviour. Kısa ses efektleri + cihaz titreşimi. Eksik klipler atlanır. SFX → SFX toggle'ı, titreşim →

Vibration toggle'ı (bağımsız). Klipler: `placeClip/mergeClip/clearClip/invalidClip/gameOverClip`.

`Bind(settings)`, `PlayPlace()`, `PlayGameOver()`, `PlayMerge()/PlayClear()/PlayInvalid()` (klip + `Vibrate()`).

`PlayFor(result)` — `HasLineClear` → `Clear` (merge'e üstün), `HasMerge` → `Merge`, aksi halde `Place`.

`Play(clip)` — `null/!SfxEnabled` ise çık, değilse `PlayOneShot`. `Vibrate()` — `#if UNITY_ANDROID || UNITY_IOS`

altında `VibrationEnabled` açıksa `Handheld.Vibrate()` (platform guard'ı; korumasız çağrı standalone/WebGL'de derleme hatası).

12. Testler (Test Katmanı)

Proje, klasik bir **test piramidi** kullanır: hızlı ve izole EditMode testleri tabanı, gerçek sahneyi süren PlayMode testleri tepeyi oluşturur. Toplam **25 EditMode + 5 PlayMode** testi vardır (Unity Test Runner: Window ► General ► Test Runner).

MoveResolutionTests.cs

Namespace: `NumbersBlast.Tests` · **Tür:** EditMode (headless / CI'da çalışır, asset gerektirmez).

Oyunun kalbi olan **deterministik hamle çözümüleme** hattını test eder: `PlacementService` (+ `MergeResolver`, `LineClearResolver`), `ScoreService`, `PieceFactory`, `BoardModel`.

Yardımcı metotlar:

- `NewPlacement()` — `new PlacementService(new MergeResolver(), new LineClearResolver())` kurar.
- `SingleCell(int value)` — tek hücreli `PieceInstance` üretir.
- `SetPrivate(...)` — Reflection ile private alanları doldurur (ScriptableObject'i diskten yüklemekten kurmak için).
- `BuildConfig()` — 2x2 kare şekilli, 1..4 değer aralıklı bir `PieceSetConfig`'i tamamen kod içinde inşa eder.

Test metotları:

1. `CanPlace_RejectsOutsideAndOccupied` — (0,0) geçerli; tahta dışı (-1,0)/(8,0) ve dolu hücre reddedilir.
2. `Merge_ThreeOnes_BecomesThree` — Üç "1" birleşip "3" olur; sonuç geçerli, `HasMerge` doğru.
3. `Merge_ChainReaction_OnesBecomeThreeThenSix` — Zincir: iki 1 önce 3'e, sonra doğudaki 3 ile birleşip 6 olur; zincire giren hücre boşalır.
4. `LineClear_Row_ScoreEqualsSumOfClearedValues` — Satır temizlenir; `ScoreGain` temizlenen değerlerin toplamına eşit.
5. `LineClear_Column_Clears` — Aynı mantığın dikey (sütun) versiyonu.
6. `LineClear_RowColumnIntersection_CountedOnce` — Satır + sütun aynı anda temizlenirse kesişim hücresi skorda **bir kez** sayılır: `expected = (2+3+2) + (3+2+3) + 4`.
7. `Preview_OnScratchBoard_DoesNotMutateRealBoard` — WYSIWYG garantisi: "scratch" tahtaya `ApplyMove` gerçek tahtayı değiştirmez.
8. `InvalidMove_LeavesBoardUntouched` — Geçersiz hamle: `IsValid` false, `ScoreGain` 0, eski değer korunur.
9. `ScoreService_PenaltyFloorsAndClampsToZero` — `Add(47) → ApplyPenaltyPercent(0.05f) = floor(2.35)=2 → 45`; küçük skorda ceza 0'a yuvarlanır, sıfır altına inmez.
10. `HasAnyValidMove_FullBoard_ReturnsFalse` — Dolu tahtada geçerli hamle yok.
11. `HasAnyValidMove_OneFreeCell_ReturnsTrue` — Tek boş hücre varsa tek hücreli parça sığar.
12. `HasAnyValidMove_MultiCellPiece_NeedsContiguousSpace` — Çok hücreli parça bitişik boş alan ister (ayrık delikler yetmez).
13. `HasAnyValidMove_NullPiecesInTray_AreSkipped` — Tepsideki `null` parçalar atlanır.

14. **PieceFactory_NeverSpawnsAdjacentEqualValues** — 300 iterasyon boyunca üretilen parçalarda yan yana hücreler asla aynı değere sahip olmaz (seedli RNG: 987654321).

Rol: Merge, zincir, satır/sütun temizleme, skor, ceza, önizleme ve geçersiz-hamle korumasının sözleşmesini kilitler; kabul kriterlerine birebir karşılık gelir. Ayrı bir **PieceFactory** test dosyası yoktur; fabrika davranışı buradaki **PieceFactory_NeverSpawnsAdjacentEqualValues** ile test edilir.

GameFlowPlayModeTests.cs

Namespace: NumbersBlast.PlayModeTests · **Tür:** PlayMode ([UnityTest] + IEnumerator, gerçek "Game" sahnesini yükler).

Uçtan uca oyun akışını "smoke test" olarak sürer: **GameSessionController**, tepsi, Solo/Vs AI modları, tur zamanlayıcı, girdi kilidi. Simüle fare/sürükleme yerine kontrollü oturum API'si kullanır.

SetUp/TearDown: SetUp tutorial'ı "görüldü" işaretler (**PlayerPrefs**) ki ilk solo turda eğitim kapısı atlansın; TearDown **Time.timeScale = 1** yapar ve gerçek tutorial anahtarını geri yazar.

Test metotları:

1. **Scene_Loads_WithSessionPresent** — Sahne yüklenince **GameSessionController** mevcut.
2. **Solo_Starts_ThenAValidMoveResolves** — Solo başlar, tepsi 3 parça; geçerli hamle bulunur, **ApplyPlayerMove** uygulanır, resolve+refill sonrası tepside parça vardır.
3. **VsAI_Starts_AndReachesPlayerTurn** — Vs AI başlar, **Time.timeScale = 5** ile matchmaking hızlanır, oyuncu sırası (**CanAcceptPlayerInput**) 8 sn içinde gelir.
4. **VsAI_PlayerTimeout_PassesTurnToOpponent_ThenBackToPlayer** — Regresyon: oyuncu hiç oynamaz, 20 sn dolar, sıra rakibe geçer (rakip görünür şekilde oynar), sonra oyuncuya döner. Taraf-etiketli **MoveResolved** düzeltmesini kapsar.
5. **InputLock_FromInGameSettings_ClearsWhenReturningToMenu** — Regresyon: ayarlardan menüye dönerken takılı kalan girdi kilidinin temizlendiğini doğrular.

Yardımcı: **CountOccupied(BoardModel)** — tahtadaki dolu hücreleri sayar.

Rol: EditMode parçaları izole doğrularken bu dosya gerçek sahnede entegrasyonu ve geçmiş regresyonları (timeout, girdi kilidi) doğrular.

OpponentActPlannerTests.cs

Namespace: NumbersBlast.Tests · **Tür:** EditMode (seedli **System.Random**, deterministik, headless).

OpponentActPlanner'ı — rakibin karar/"beat" planlayıcısını — test eder. Çıktı bir **OpponentBeat** listesidir (SelectDelay, TravelTo, Misdrop, SwapFakeout, FinalPlace).

Yardımcılar: **NewPlacement()**, **OneCell(int)**, **Travels(beats)** (yalnız **TravelTo** hücrelerini süzer).

Test metotları:

1. **Planner_Decoys_AreDistinctValidAndNeverTheAnchor** — Son **TravelTo** gerçek çapa; öncekiler decoy: geçerli, birbirinden farklı ve asla çapa değil.
2. **Planner_Misdrop_TargetsAGenuinelyInvalidCell** — Misdrop şansı 1f: rakip dolu/geçersiz bir hücreye "insan gibi" yanlış bırakma dener.
3. **Planner_BeatCount_IsBoundedAndEndsWithFinalPlace** — Beat sayısı 5..25; ilk **SelectDelay**, son **FinalPlace** (çapada). 0 denemeli tur: tek **TravelTo** doğrudan çapaya.

4. `Planner_SwapFakeout_GrabsAnotherPieceThenStillPlaysTheRealAnchor` — Swap şansı `1f`: başka parça alıp bırakır gibi yapar ama yine gerçek çapada `FinalPlace` ile biter.

Rol: Rakibin insansı davranış katmanını (yanıltma, yanlış bırakma, parça değiştirme) determinist olarak kilitler; sınır dışına çıkmaz, hep geçerli gerçek hamleyle sonlanır.

PlayerProgressTests.cs

Namespace: `NumbersBlast.Tests` · **Tür:** `EditMode`. Test key prefix'i (`test_nbpp_`) kullanır, `TearDown`'da siler — gerçek oyuncu verisine dokunmaz.

`PlayerProgress`'i (en iyi skor + tutorial-görüldü, `PlayerPrefs` üzerinden kalıcı) test eder. Yardımcı: `Fresh()` prefix ile yeni örnek yaratıp `Load()` çağırır.

Test metotları:

1. `PlayerProgress_TrySubmitBestScore_RejectsVsAI` — Vs AI skoru en iyi skora yazılmaz.
2. `PlayerProgress_TrySubmitBestScore_RejectsTutorial` — Tutorial aktifken skor reddedilir.
3. `PlayerProgress_TrySubmitBestScore_RejectsLowerScore` — 50 kabul, sonra 40 reddedilir, en iyi 50 kalır.
4. `PlayerProgress_MarkTutorialSeen_Persists` — Tutorial-görüldü işareti reload'dan sağ çıkar.

Rol: Sadece meşru solo skorların en iyi skor olmasını, hile modlarının sayılmamasını ve tutorial durumunun kalıcılığını garanti eder.

TrayModelTests.cs

Namespace: `NumbersBlast.Tests` · **Tür:** `EditMode`, `headless`. Görünümden bağımsız, oturumun sahip olduğu **otoriter** tepsi modelini test eder.

`TrayModel`'i test eder. Yardımcılar: `Piece(int)`, `ContainsRef(...)` (referans/kimlik kontrolü, değer eşitliği değil).

Test metotları:

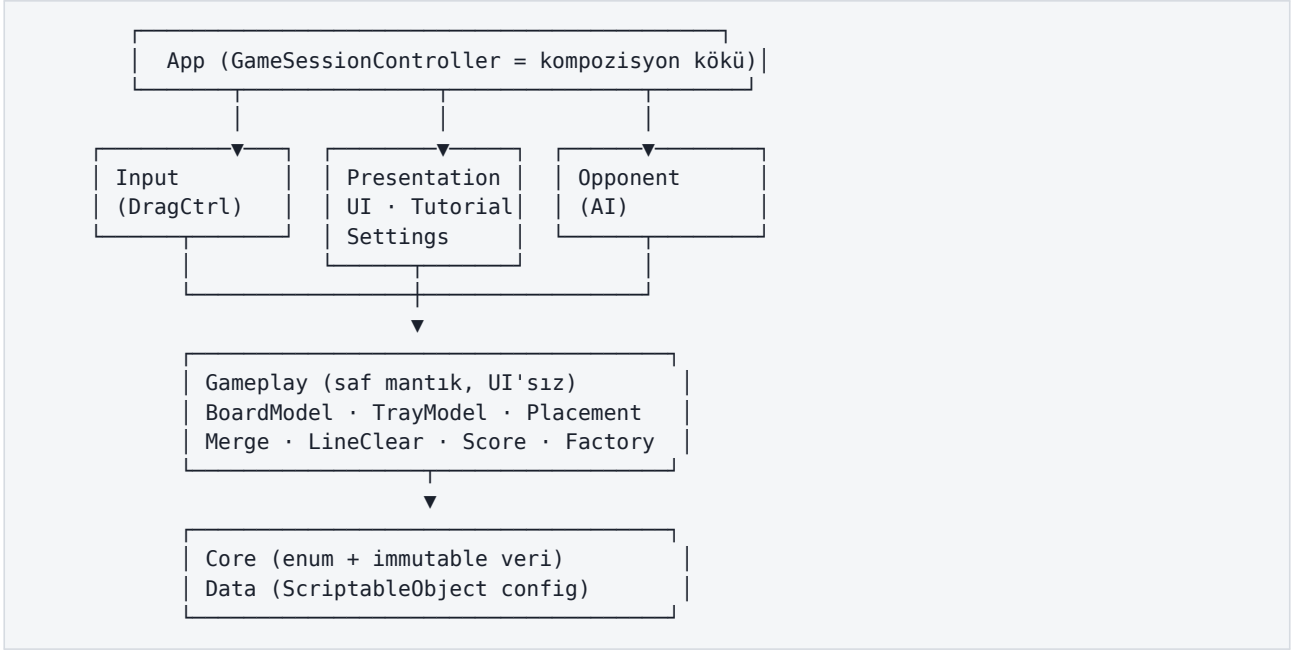
1. `TrayModel_SetPieces_ReplacesAll` — `SetPieces` tüm parçaları değiştirir; doğru referans tutulur.
2. `TrayModel_Remove_RemovesOnlySelectedPiece` — `Remove` yalnız seçileni çıkarır, diğerleri referansça kalır.
3. `TrayModel_IsEmpty_TracksCount` — `IsEmpty` sayımı doğru izler.

Rol: Tepsi verisinin görünümden ayrı, güvenilir tek kaynak olmasını sağlar; sürükle-bırakta yanlış parçanın tüketilmesini önler.

13. Genel Mimari Özet ve Öğretici Çıkarımlar

Tüm 58 dosyayı katman katman inceledik. Şimdi projenin bütününe birbirine bağlayan desenleri toparlayalım.

Katmanlar Arası Bağımlılık Yönü



Ok yönü "referans eder"i gösterir. **Alt katmanlar üstü asla referans etmez.** Gameplay hiçbir UI/Input

tipini tanımaz; bu, saf mantığın headless test edilebilirliğini garanti eder (25 EditMode testi bunu kullanır).

Tekrarlanan Tasarım Desenleri

Desen	Nerede	Neden
Tek doğruluk kaynağı	<code>PlacementService.ApplyMove</code> , <code>PiecePlacementPreview.Tick</code>	Önizleme, gerçek hamle ve AI tek koddan → yapısal ayrışma imkânsız
Immutability (değişmezlik)	<code>MoveResult</code> , <code>MergeStep</code> , <code>PieceInstance</code> , <code>PieceCellData</code>	Yan etkisiz, güvenli paylaşım; simülasyonda güven
Scratch board (CopyFrom)	Önizleme + AI değerlendirmesi	Gerçek tahta bozulmadan deneme hamleleri
Observer (event)	<code>ScoreService.ScoreChanged</code> , <code>GameSettings.Changed</code> , UI Action event'leri	UI/ses modeli sorgulamak yerine değişimi dinler; gevşek bağ

Desen	Nerede	Neden
Kompozisyon kökü	<code>GameSessionController.Awake</code>	Tüm bağımlılıklar tek noktada <code>new</code> 'lenir; DI framework yok
Model/View ayrımı	<code>TrayModel</code> vs <code>PieceTrayView</code> , <code>BoardModel</code> vs <code>BoardView</code>	Yetkili veri modelde; view yalnız çizer
Allocation optimizasyonu	<code>MoveResult</code> boş diziler, glow havuzu, GC'siz önizleme	AI çok sayıda hamle değerlendirirken GC baskısını azaltır
Deterministik RNG (seed)	<code>PieceFactory</code> , <code>OpponentMoveEvaluator</code> , <code>OpponentActPlanner</code>	Tekrarlanabilir/test edilebilir davranış
Savunmacı sınır kontrolü	<code>BoardModel</code> tüm erişimlerde <code>IsInside</code>	Sınır dışı sessizce "boş" → algoritmalar basitleşir

Bir Hamlenin Uçtan Uca Yolculuğu

- Üretim:** `PieceFactory.CreateRandomPiece()` → komşu-farklı-değer kuralıyla parça → `TrayModel` + `PieceTrayView`.
- Sürükleme:** `PieceDragController.OnDrag` → `PiecePlacementPreview.Tick` → `BoardView` scratch üzerinde `PreviewMove` sonucunu boyar.
- Bırakma:** `OnPointerUp` → `CanSubmitMove` (üç kapı) → `ApplyPlayerMove` → `ApplyMoveInternal` (model+view birlikte tüketilir).
- Çözümleme:** `PlacementService.ApplyMove` = Yerleştir → Merge (BFS+toplama+zincir) → Line Clear (HashSet tek-sayım).
- Sunum:** `GameplayAnimationController.PlayMove(result)` → pop/merge-uçuş/clear-parlama/uçan skor. `SfxPlayer.PlayFor(result)`.
- Skor:** `MoveResult.ScoreGain` → `ScoreService.Add` → `ScoreChanged` → `ScoreView`. Solo'da `PlayerProgress.TrySubmitBestScore`.
- Sıra/Fail:** Solo → `HasAnyValidMove` false ise `SoloGameOver`. VsAI → `TurnController.MatchLoop` sırayı rakibe geçirir.

VS AI Modunun Dört İlkesi (Özet)

- Oyuncuyla aynı pipeline** — AI, `PlacementService.ApplyMove` ile aynı kurallarla değerlendirir; ayrı motor yok.
- Rubber-band** — `SelectionWidth(scoreDelta)`: önde gevşer (3 aday), geride sıkışır (1 aday); `ObviousMoveGap=25` ile bariz hamle asla kaçmaz.
- İnsansı, dürüst sunum** — tereddüt gerçek karar belirsizliğinden (`LastTopGap` → `thinkScale`); hiçbir decoy gerçek hamleden iyi görünen hücreyi göstermez; idatif el-hareketi robotluğu giderir.
- 20 sn sayaç + görünür ceza** — `TurnTimer` duraklatılamaz; gerçek timeout %5 ceza + flash; aynı-kare çıkışmasında hamle kazanır.

Sonuç

Numbers Blast, küçük ama disiplinli bir kod tabanıdır: **tek assembly, klasör=namespace, tek yönlü bağımlılık,

tek doğruluk kaynağı.** DI framework'ü, service locator, event bus veya kullanılmayan interface yoktur — her

sınıfın rolü ve bağımlılığı düz ve izlenebilirdir. Saf mantık (Core/Gameplay) UI'dan tamamen ayrıldığı için

hem headless test edilebilir hem de önizleme/gerçek/AI senaryolarında birebir tutarlıdır. Bu doküman, projeyi

inceleyecek bir geliştirici için her dosyanın "ne, neden ve nasıl"ını uçtan uca kapsar.